

Lab 3: Bayesian Learning and Boosting

Lucas Frykman

March 2, 2026

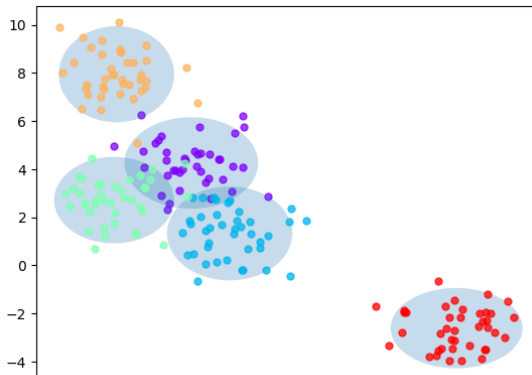
Assignment 1 Bayesian Learning

`mlParams(X, labels)` outputs the following MLE estimate

$\operatorname{argmax} p(\mu_k, \sigma_k | D_k)$

$$\mu_k = \frac{\sum_{i|c_i=k} x_i}{N_k}$$

$$\Sigma_k = \frac{1}{N_k} \sum_{i|c_i=k} (x_i - \mu_k)^T (x_i - \mu_k)$$



Assignment 2 Classification

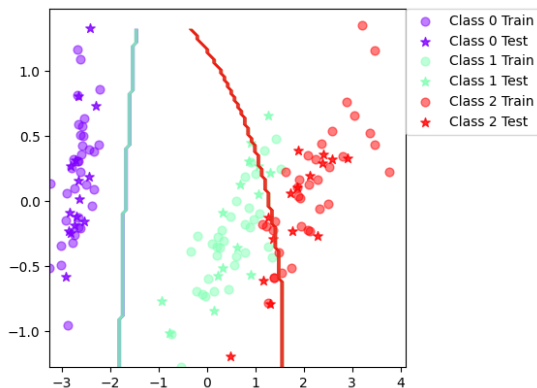
`computePrior(X, labels)` outputs a $C \times 1$ array $p(k) \approx \frac{N_k}{N}$

`classifyBayes(X, prior, mu, sigma)` outputs a $N \times 1$ array of predictions $\arg\max_k \delta_k(x)$ of test data x

where the discriminant is the log posterior

$$\delta_k(x) = \ln(p(k|x)) = -\frac{1}{2} \ln(|\Sigma_k|) - \frac{1}{2}(\mathbf{x}^* - \boldsymbol{\mu}_k)\Sigma_k^{-1}(\mathbf{x}^* - \boldsymbol{\mu}_k)^\top + \ln(p(k)) + C.$$

Assignment 3: Naive Bayes Analysis



In the figure above illustrates the boundary lines $\{x : \delta_i(x) = \delta_j(x)\}$
Final mean classification accuracy: 89 with standard deviation 4.16

Assignment 3: Naive Bayes Analysis

1. When can a feature independence assumption be reasonable and when not?

Reasonable when:

- Physically independent features (eg: petal length and time of day)
- Limited training data: Using complex models that do not hold the independence assumption risk overfitting when learning feature dependencies
- When using a ranking based classifier like naive bayes. Ranking is maintained even if independence assumption is wrong.

Unreasonable when: Physically dependent features (eg: edjacent pixels on a image)

Assignment 3: Naive Bayes Analysis

2. how does the decision boundary look for the Iris dataset? How could one improve the classification results for this scenario by changing classifier or, alternatively, manipulating the data?

- The decision boundary between class 0 and class 1 is straight due to large gap
- The decision boundary between class 1 and class 2 is curved and is underfit
- The data isn't uncorrelated (petal length and petal width tend to correlate well).
- Possible fix 1: use full bayes classification with full Σ_k to allow Guassian distributions that tilt
- Possible fix 2: PCA can transform the data in to uncorrelated data before feeding in to naive bayes
- Possible fix 3: Dimension reduction: multiply petal length and width together so you only have to classify based on one attribute

Assignment 4

Extend `mlParams` to `mlParams(X,labels,W)` that handles weighted instances

$$\mu_k = \frac{\sum_{\{i|c_i=k\}} \omega_i x_i}{\sum_{\{i|c_i=k\}} \omega_i}$$

$$\Sigma_k(m, m) = \frac{1}{\sum_{\{i|c_i=k\}} \omega_i} \sum_{\{i|c_i=k\}} \omega_i (x_i(m) - \mu_k(m))^2, m = 1, 2 \dots d$$

We're taking a weighted average. Some outliers in the training data need to have a bigger weight.

Assignment 5: Boosting the Bayes Classifier

- Modify computePrior to `computePrior(labels,W)` $p(k) = \frac{\sum_{i|c_i=k} w_i}{\sum w_i}$
- Implement Adaboost algorithm
- Design function `classifyBoost(X, classifiers, alphas, Nclasses)`

Assignment 5: Boosting the Bayes Classifier

- 1: **Initialize** all weights uniformly: $\omega_i^1 = \frac{1}{N}$ for $i = 1, \dots, N$
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Train weak learner using distribution ω^t
- 4: Get weak hypothesis h^t and compute its error ϵ^t with respect to ω^t :

$$\epsilon^t = \sum_{i=1}^N \omega_i^t (1 - \delta(h^t(x_i), c_i))$$

- 5: Choose the classifier vote weight α^t :

$$\alpha^t = \frac{1}{2} (\ln(1 - \epsilon^t) - \ln(\epsilon^t))$$

- 6: Update the training instance weights:

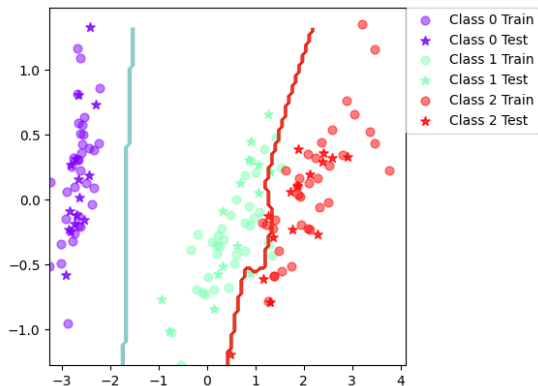
$$\omega_i^{t+1} = \frac{\omega_i^t}{Z^t} \times \begin{cases} e^{-\alpha^t} & \text{if } h^t(x_i) = c_i \\ e^{\alpha^t} & \text{if } h^t(x_i) \neq c_i \end{cases}$$

(where Z^t is a normalization factor ensuring $\sum_i \omega_i^{t+1} = 1$)

- 7: Classifier:

$$H(x) = \arg \max_{c_i} \sum_{t=1}^T \alpha^t \delta(h^t(x), c_i)$$

Assignment 5: Boosting the Bayes Classifier



Assignment 5: Boosting the Bayes Classifier

Dataset	Basic Bayes	Boosted Bayes
Iris	Acc:89.0 std:4.16	Acc:94.7 std:2.82
Vowels	Acc:64.7 std:4.03	Acc:80.2 std:3.52

Table: Mean classification accuracy and standard deviation for the Basic and Boosted Bayes classifiers.

1. Is there any improvement in classification accuracy? Why/why not?

Yes, there is an improvement.

This happens because the boosting algorithm actively re-weights misclassified examples so that they carry more importance in the next round (iteration $t+1$) of training. By doing this, the model is forced to focus harder on the difficult points and classify them correctly in subsequent runs

Assignment 5: Boosting the Bayes Classifier

2. Plot the decision boundary of the boosted classifier on iris and compare it with that of the basic. What differences do you notice? Is the boundary of the boosted version more complex?

- Performance difference: The regular classifier incorrectly classified some of the red and green data points, while the boosted version successfully captured and classified them. This improvement is a direct result of the algorithm punishing those wrong classifications and increasing their weights.
- Yes the boosted bayes boundary is more complex and is wraps around specific training data. Bias has been decreased.

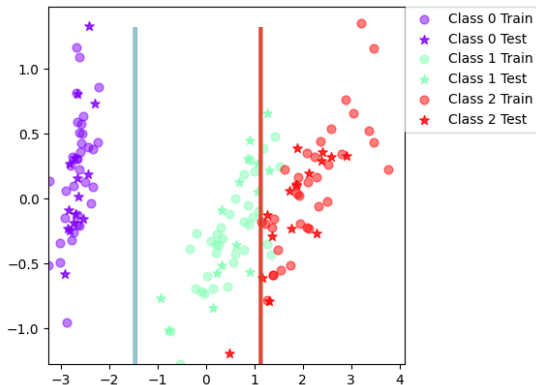
Assignment 5: Boosting the Bayes Classifier

3. Can we make up for not using a more advanced model in the basic classifier by using boosting? Yes, mostly. Boosting effectively compensates for a simple base classifier because the ensemble as a whole acts like a much more advanced and capable model.

The Catch: You have to be careful, as this increased complexity can easily cause the model to overfit the training data (and its noise), meaning boosting might not always be the best option depending on the dataset.

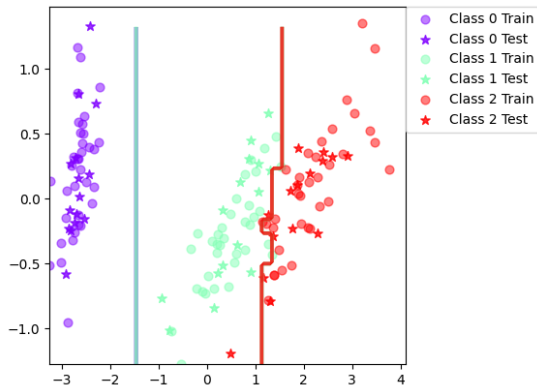
Assignment 6: Decision Trees vs. Boosting

Decision tree boundary without boosting



Assignment 6: Decision Trees vs. Boosting

Decision tree boundary with boosting



Assignment 6: Decision Trees vs. Boosting

Dataset	Basic Decision Tree	Boosted Decision Tree
Iris	Acc:92.4 std:3.67	Acc:94.6 std:3.67
Vowels	Acc:64.1 std:4.02	Acc:86.7 std:2.95

Table: Mean classification accuracy and standard deviation for the Basic and Boosted Decision Tree classifiers.

Assignment 6: Decision Trees vs. Boosting

1. Is there any improvement in classification accuracy? Why/why not?

- Yes this occurs because Adaboost actively increases the weight of misclassified data points after every training round.
- The next classifier focuses entirely on the hardest examples correcting the ensembles previous mistakes

Assignment 6: Decision Trees vs. Boosting

2. Plot the decision boundary of the boosted classifier on iris and compare it with that of the basic. What differences do you notice? Is the boundary of the boosted version more complex?

- Yes the boosted decision boundary is more complex.
- The regular boundary misclassified some green points (higher bias)
- The boosted boundary bends to achieve both low training and test error (low variance and low bias)

Assignment 6: Decision Trees vs. Boosting

3. Can we make up for not using a more advanced model in the basic classifier by using boosting? Yes, because the boosting will act like a more advanced model, but it can cause overfitting so it might not always be the best option.

Assignment 7: Model Selection Criteria

Based on the lab criteria, here is how the classifiers compare

- **Outliers:** Decision Trees isolate outliers well. Naive Bayes means are easily skewed by them (if too little training data)
- **Irrelevant Inputs:** Trees perform inherent feature selection. Naive Bayes assumes all features are equally relevant
- **Predictive Power:** Boosted Trees typically offer the highest overall predictive accuracy
- **Mixed Data:** Trees handle categorical/mixed data natively. Gaussian Bayes requires continuous distributions
- **Scalability:** Naive Bayes scales exceptionally well with large dimensions (D) and instances (N). Trees become expensive as D grows

Criterion	Naive Bayes	Boosted Naive Bayes	Decision Tree	Boosted Decision Tree
Outliers	Sensitive due to Gaussian assumption	More robust, but still sensitive to outliers causing overfitting	Robust (uses threshold checks), but outliers can form small leaves	Sensitive; misclassified outliers get overemphasized in later rounds
Irrelevant Inputs	Assumes independence, struggles with irrelevant features	Improves, but still assumes independence	Avoids splitting on them, but might choose them by chance if data is limited	Avoids them initially, but later rounds might fit to noise/irrelevant features
Predictive Power	Weak for complex data	Moderate improvement, but limited by feature independence	Good for complex boundaries, but may overfit without pruning	High predictive power; ensemble captures highly complex boundaries
Mixed Types of Data	Works only for numeric data	Same limitations as base Naive Bayes	Handles naturally (categorical and numeric thresholds)	Handles naturally, identical to the base Decision Tree
Scalability	Very scalable, works well with large datasets	Slower due to sequential boosting iterations	Handles large datasets well, but full trees can be expensive to build	Sequential building is expensive, but mitigated by using shallow trees (stumps)

Table 1: Comparison of Naive Bayes, Decision Trees, and their Boosted Versions